

RECURSION, FRACTALS AND SPACE-FILLING CURVES IN LOGO

Alina Musaeva, Julian Schwarzer, Lars Wittner
Proseminar – Advanced programming concepts

Contents

- **Logo & Turtle-Graphics**
 - **What is Logo?**
 - **Turtle Graphics**
 - **Base commands**
- **Recursion & Fractals**
 - **What is recursion?**
 - **Pythagoras tree**
 - **Fractals**
 - **Koch flake**
- **L-Systems**
 - **Production rules**
 - **Edge rewriting**
 - **Node rewriting**
 - **Branching**

Logo & Turtle-Graphics

What is Logo?

- Programming language with **turtle graphics**
- Drawing shapes
- Educational purpose
- Intuitive for beginners
- Visualisation



Logo & Turtle-Graphics History

- **Publication:** 1967, Cambridge USA
- **Developed by:** Wally Feurzeig, Seymour Papert, Cynthia Solomon
- **Based on:** Lisp
- **Alternatives:** Scratch and Python with Turtle Graphics

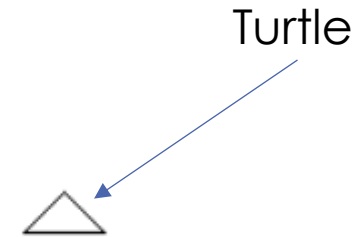
Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

Logo & Turtle-Graphics

Turtle Graphics

- Cursor or pen (turtle)
- Executing commands from console line by line
- Drawing after a command on the computer



Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

Logo & Turtle-Graphics

Base Commands

FORWARD | BACK *length*
LEFT | RIGHT *degree*

```
forward 50  
back 50  
left 45  
right 45
```

CLEARSCREEN
PENUP | PENDOWN
PRINT *"word | number | [value list]*

```
print "Hello  
print [Hello world]  
print 45
```


Logo & Turtle-Graphics

Base Commands

Dynamic data types (*word, number, list, bool, ...*)

```
"word | 1, 0.5, pi | [more text] | TRUE
```

MAKE “*varname value | [value list]*”

```
make "five 5  
make "fives [5 6 7 8]
```

Logo & Turtle-Graphics

Base Commands

REPEAT *number* [*commands*]

```
repeat 360 [fd 1 rt 1]
```

TO *procedure_name* *:param1* ... *:paramN*
commands

END

```
to polygon :num  
  repeat :num [fd 100 rt 360/:num]  
end
```

Logo & Turtle-Graphics

Base Commands

IF | IFELSE *condition* [*condition true*] [*condition false*]

```
ifelse 1 < 2 [print "yes] [print "no]
```

WHILE *condition* [*commands*]

```
make "i 1  
while [:i < 4] [print :i make "i :i + 1]
```

FOR [*var start end step*] [*commands*]

```
for [i 1 10 2] [print :i]
```

Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

Recursion & Fractals

What is recursion?

- Recursion *lat. recurrere (to go back)*
- Calls itself
- Base case with termination condition
- Recursive step (approximate base case)
- Depth of recursion
- Usefull concept in mathematics (factorial, Fibonacci, ...) or computer science



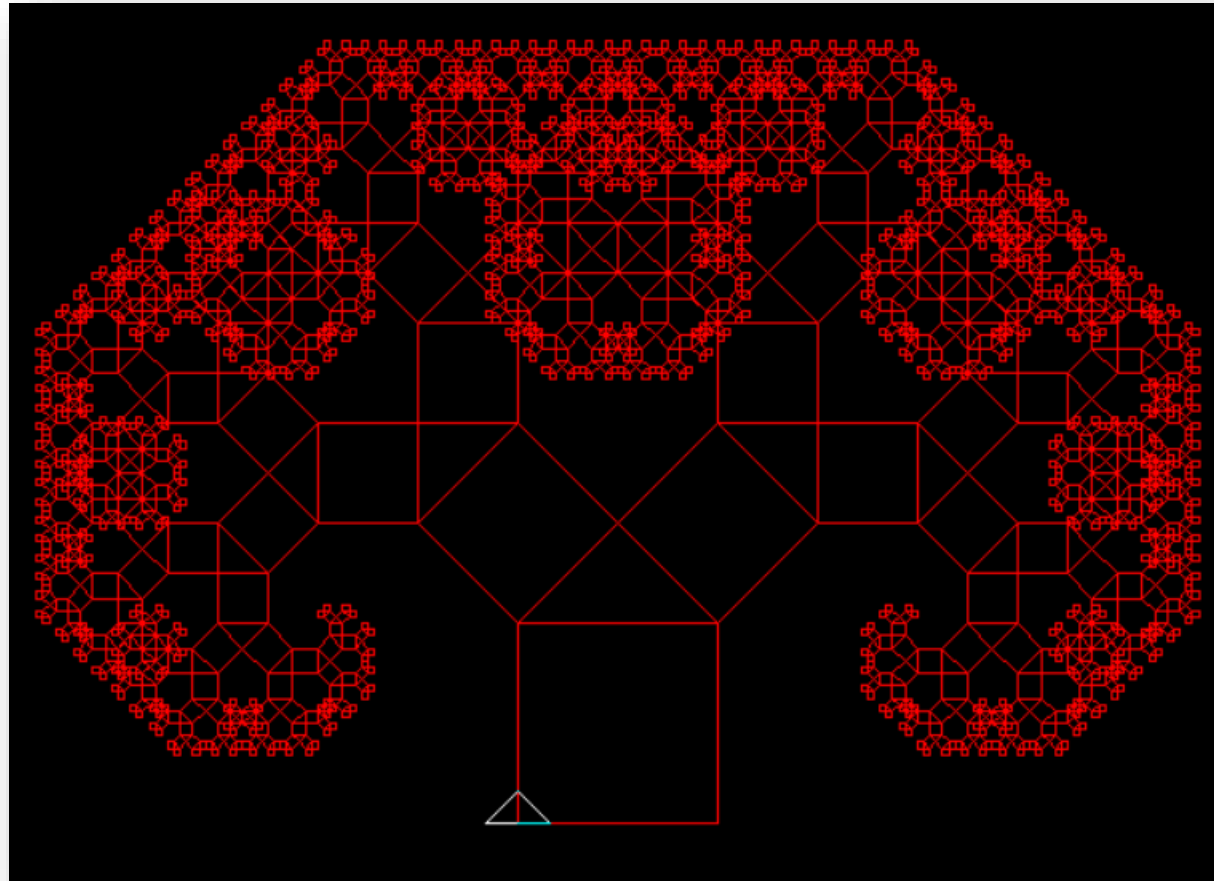
```
// return nth fibonacci number
private static int fib(int n) {
    // wrong input
    if (n <= 0) return -1;
    // base case 1
    if (n == 1) return 0;
    // base case 2
    if (n == 2) return 1;
    // recursive call
    return fib(n - 1) + fib(n - 2);
}
```

Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

Recursion & Fractals

Pythagoras tree

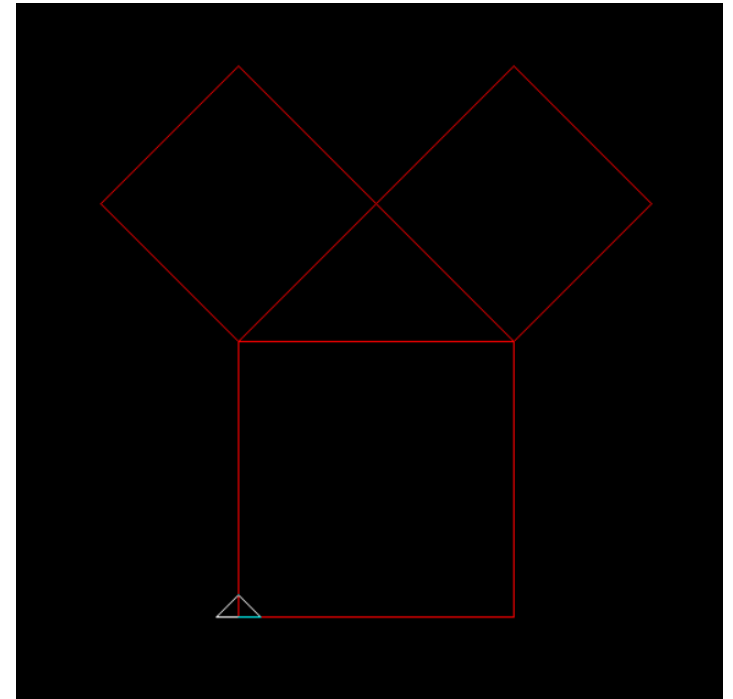


Recursion & Fractals

Pythagoras tree

```
to pythagoras :length :depth ; variable length and depth
; square
repeat 4 [fd :length rt 90]
; base case
if :depth = 0 [stop]
; change pos
fd :length lt 45
; recursive left
pythagoras :length / (sqrt 2) :depth - 1
; change pos
rt 90 fd :length / (sqrt 2)
; recursive right
pythagoras :length / (sqrt 2) :depth - 1
; go back
bk :length / (sqrt 2) lt 45 bk :length
end
```

```
bk :length / (sqrt 2) lt 45 bk 200
```

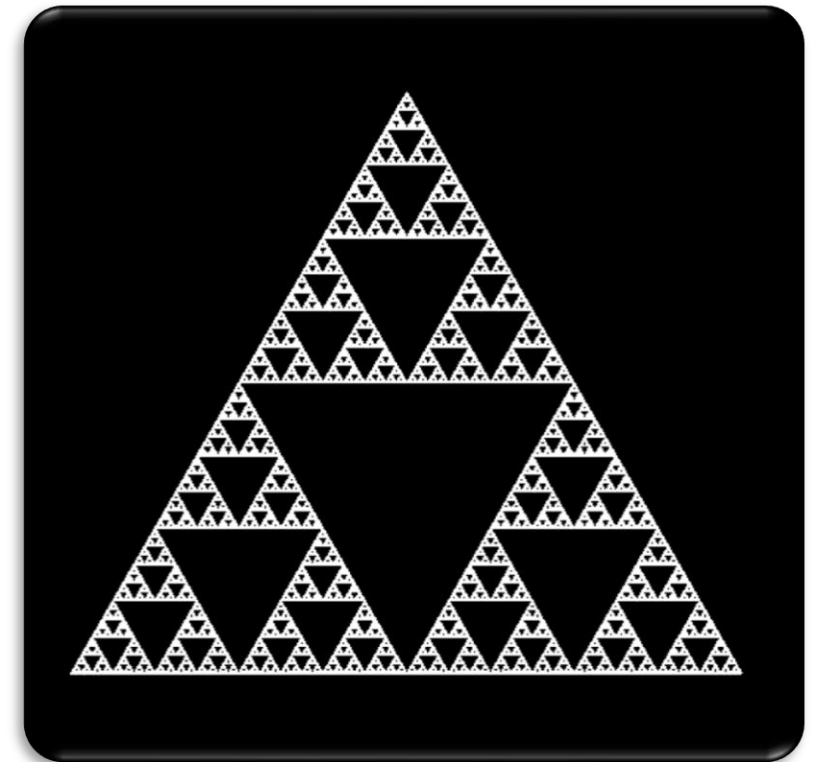
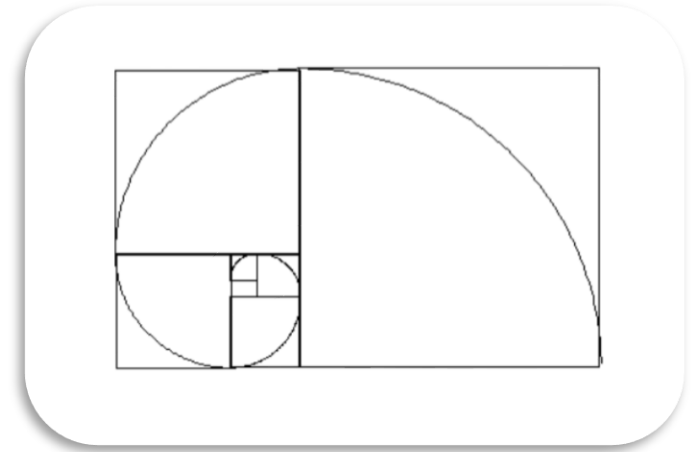


Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - **Fractals**
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

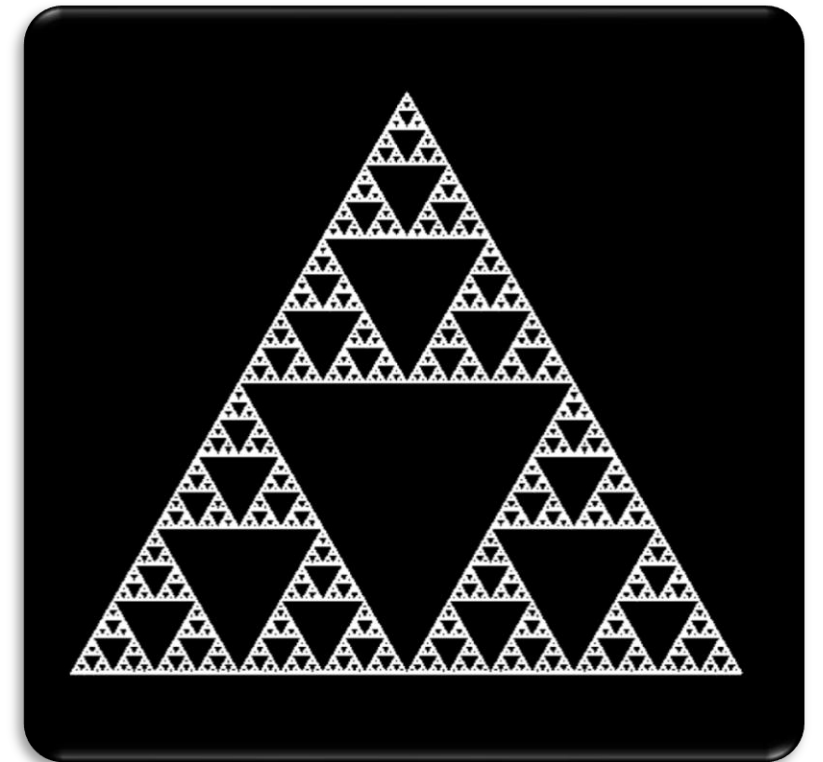
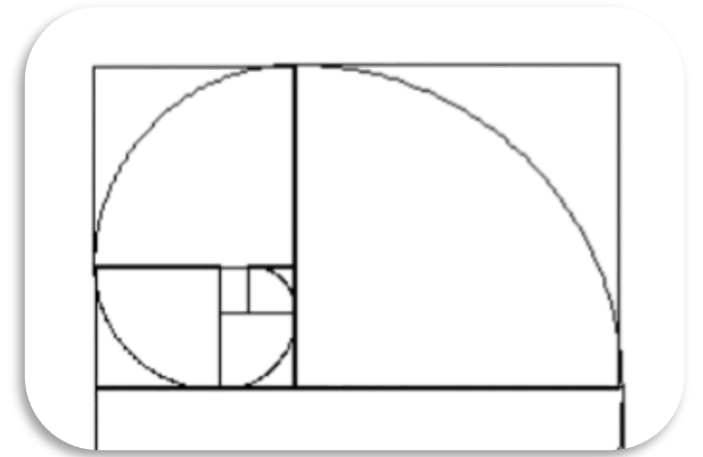
Recursion & Fractals

- The „atoms“ of recursion
- Fractal *lat. fracere (to break)*
- Recursion depth determines sharpness of picture
- Self-similar



Recursion & Fractals

- The „atoms“ of recursion
- Fractal *lat. fracere (to break)*
- Recursion depth determines sharpness of picture
- Self-similar



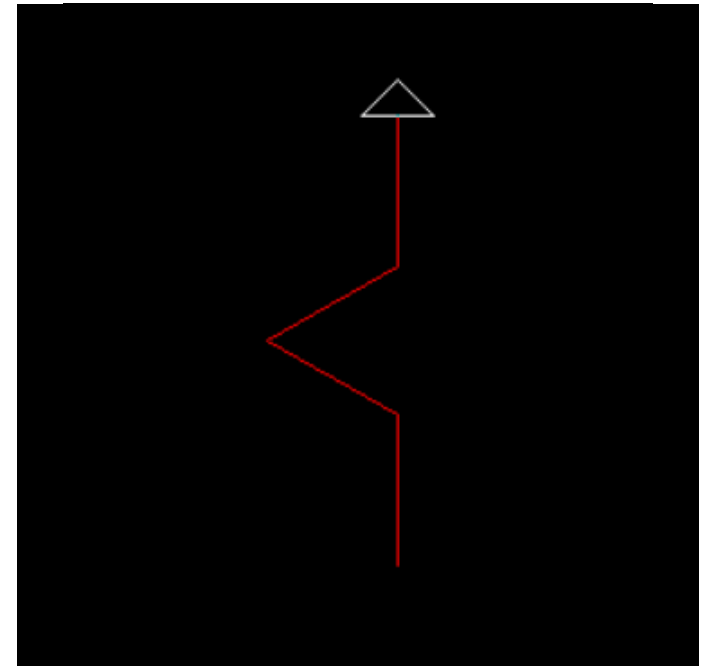
Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

Recursion & Fractals

Koch curve

```
; procedure with parameters length and depth
to koch :length :depth
  ; base case: if depth is 0, move forward and stop
  if :depth = 0 [fd :length stop]
  ; fractal part 1/4 (move)
  koch :length / 3 :depth - 1
  ; fractal part 2/4 (turn left and move)
  lt 60
  koch :length / 3 :depth - 1
  ; fractal part 3/4 (turn right and move)
  rt 120
  koch :length / 3 :depth - 1
  ; fractal part 4/4 (turn left and move)
  lt 60
  koch :length / 3 :depth - 1
end
```

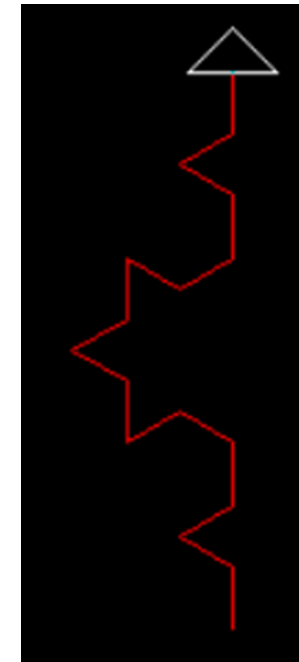


Recursion & Fractals

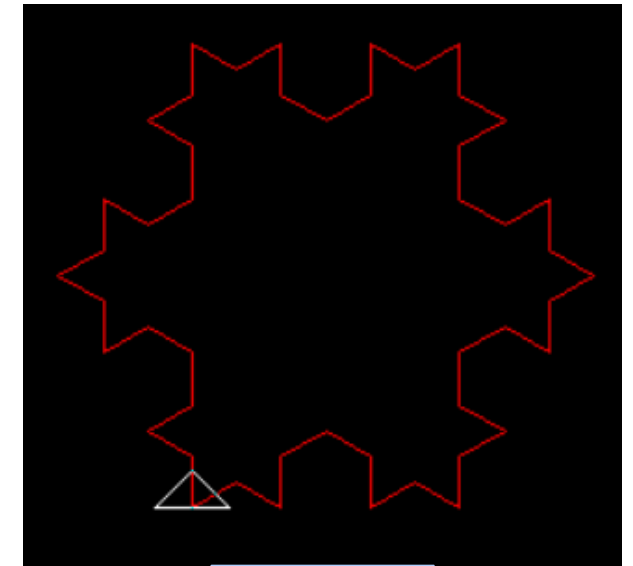
Koch flake

```
; procedure with parameters length and depth
to koch :length :depth
  ; base case: if depth is 0, move forward and stop
  if :depth = 0 [fd :length stop]
  ; fractal part 1/4 (move)
  koch :length / 3 :depth - 1
  ; fractal part 2/4 (turn left and move)
  lt 60
  koch :length / 3 :depth - 1
  ; fractal part 3/4 (turn right and move)
  rt 120
  koch :length / 3 :depth - 1
  ; fractal part 4/4 (turn left and move)
  lt 60
  koch :length / 3 :depth - 1
end

to flake :depth
  cs
  ; 3 sides of base triangle form
  repeat 3 [
    koch 200 :depth
    ; turn right to next triangle base side
    rt 120
  ]
end
```



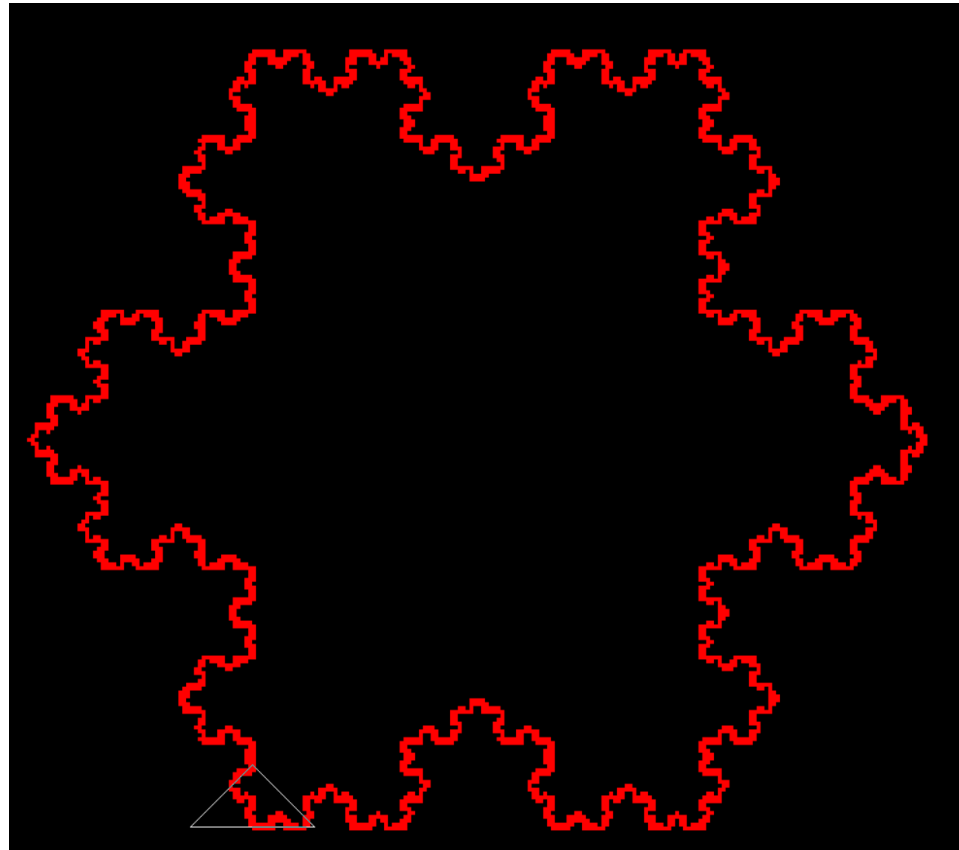
koch 200 2



flake 2

Recursion & Fractals

Koch flake



flake 7

Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

L-System Lindenmayer-System

- Aristid Lindenmayer
- Developed in 1968
- Alphabet V
- Starting word ω
- Production rules P

$\rightarrow (V, \omega, P)$

L-Systems

Production rules

- Language L
- Parallel use of production rules

Production rules

$a \rightarrow ab$

$b \rightarrow bb$

$\omega = a$ $a \in L$

$a \rightarrow ab$ $ab \in L$

$ab \rightarrow abbb$ $abbb \in L$

$abb \rightarrow abb$ $abb \notin L$

Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

L-Systems

Edge rewriting

L-System = $(\{+, -, F\}, F, P)$

$P:$
 $F \rightarrow +F - F$

parallel replacement

In Logo:
+ = rt
- = lt
F = fd

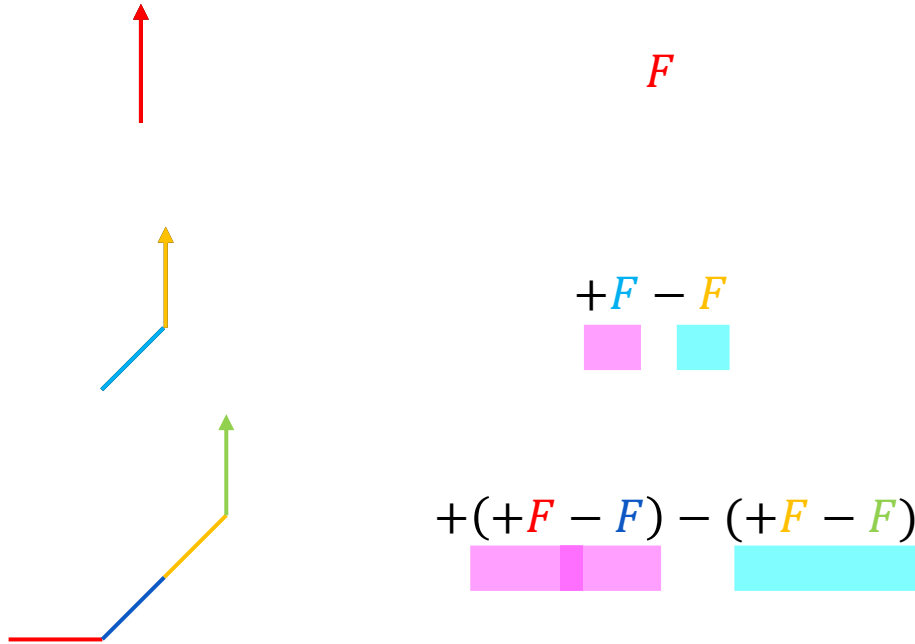
Angle 45°

L-Systems

Edge rewriting

L-System = $(\{+, -, F\}, F, P)$

$\omega = F$



parallel replacement

$P:$

$F \rightarrow +F - F$

In Logo:

$+ = \text{rt}$

$- = \text{lt}$

$F = \text{fd}$

Angle 45°

Contents

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

L-Systems

Node rewriting

L-System = $(\{+, -, F\}, FA, P)$

P :

$A \rightarrow +AF - AF$

In Logo:

$+ = \text{rt}$

$- = \text{lt}$

$F = \text{fd}$

Angle 45°

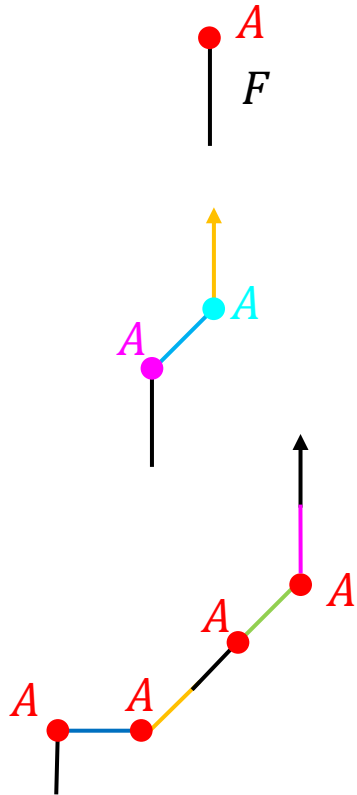
A = non-terminal

L-Systems

Node rewriting

$$\text{L-System} = (\{+, -, F\}, FA, P)$$

$$\omega = FA$$



FA

$F(+AF - AF)$

$F(+(+AF - AF)F$
 $- (+AF - AF)F)$

parallel replacement

$P:$

$A \rightarrow +AF - AF$

In Logo:

$+ = \text{rt}$

$- = \text{lt}$

$F = \text{fd}$

Angle 45°

$A = \text{non-terminal}$

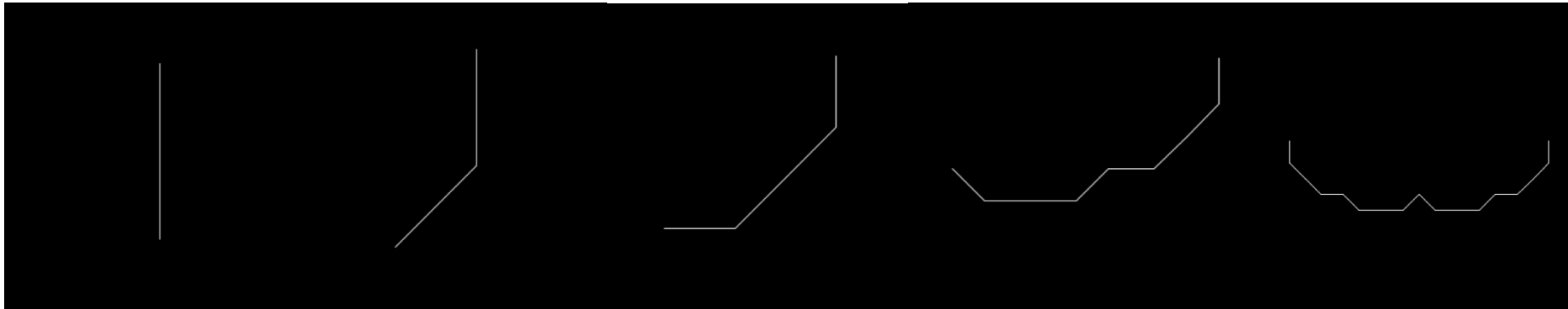
L-Systems

Examples

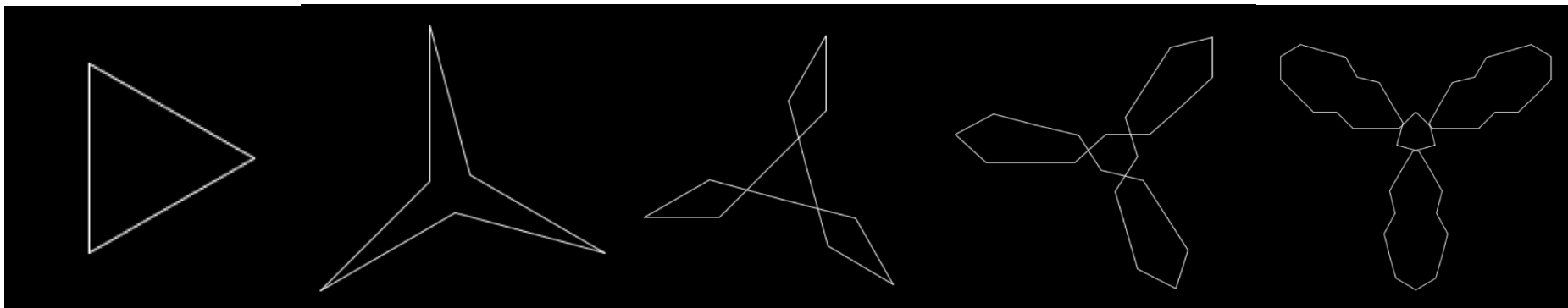
$$\omega = F$$

$$F \rightarrow +F - F$$

Angle 45°



$$\omega = F + F + F +$$



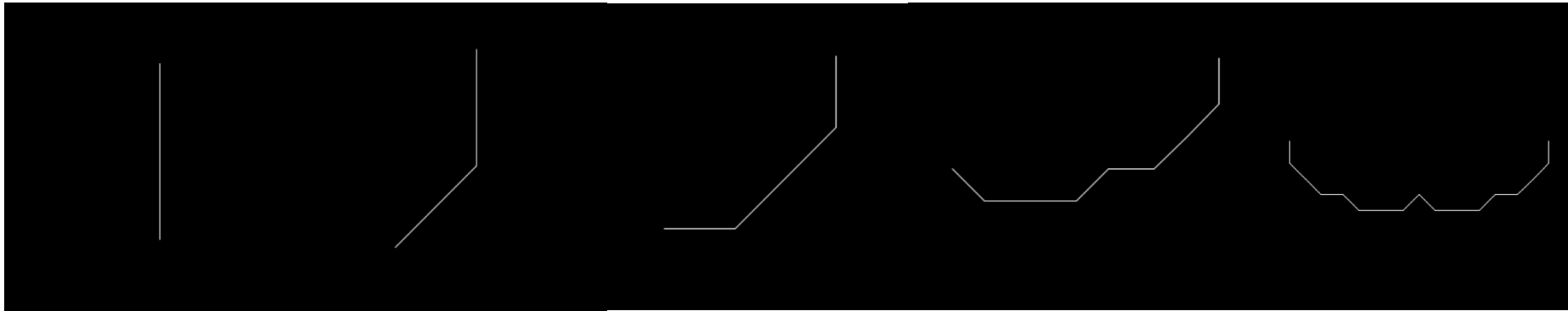
L-Systems

Examples

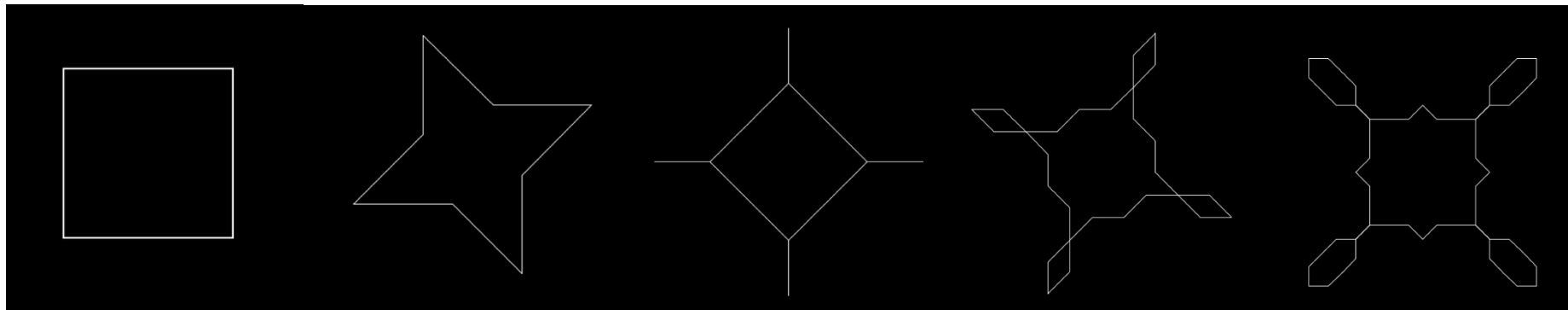
$$\omega = F$$

$$F \rightarrow +F - F$$

Angle 45°



$$\omega = F + F + F + F +$$



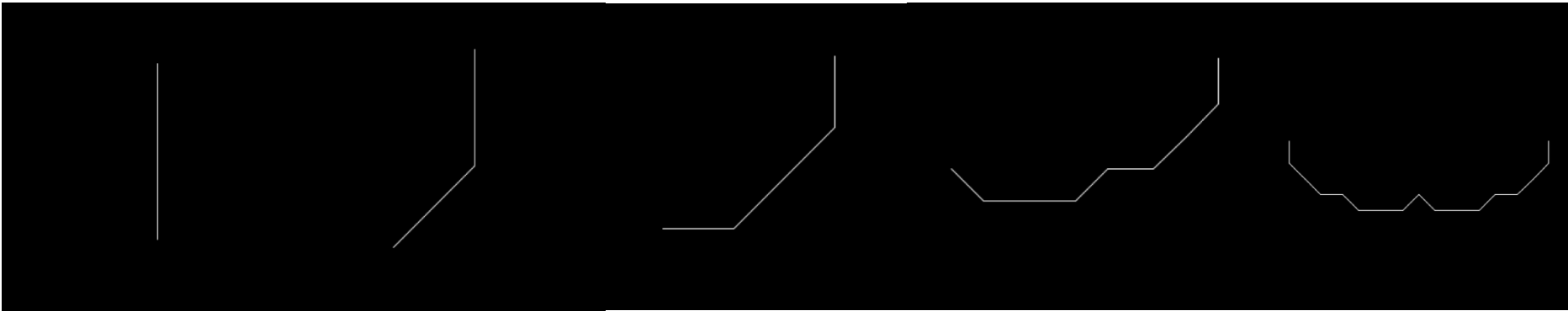
L-Systems

Examples

$$\omega = F$$

$$F \rightarrow +F - F$$

Angle 45°



$$\omega = F + F + F + F +$$

$$F + F + F + F +$$



Contents

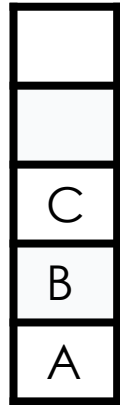
- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

L-Systems Branching

Recap: Stack

Data structure

„LIFO“



push: A, B, C

pop: C, B, A

L-Systems

Branching

- $[,] \in V$
- “[“ = push
- “]“ = pop

L-Systems Branching

Edge rewriting L-system = $(\{+, -, F, [,]\}, F, P)$

$P:$
 $F \rightarrow F[-F][+F]F$

“[“ = push
“]“ = pop

$$\omega = F$$

L-Systems Branching

Edge rewriting

L-system = $(\{+, -, F, [,]\}, F, P)$

parallel replacement

$P:$

$F \rightarrow F[-F][+F]F$

“[“ = push
“]“ = pop

In Logo:
+ = rt
- = lt
F = fd

Angle 45°

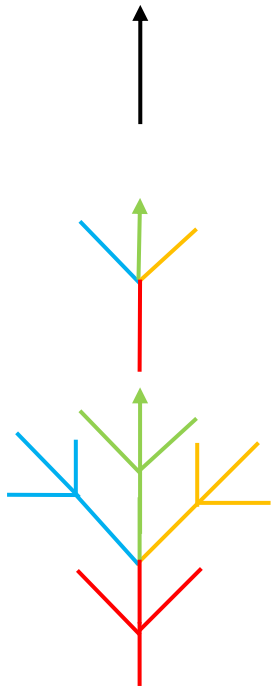
$\omega = F$

F

$F[-F][+F]F$

$F[-F][+F]F \quad [-F[-F][+F]F]$

$[+F[-F][+F]F] \quad F[-F][+F]F$



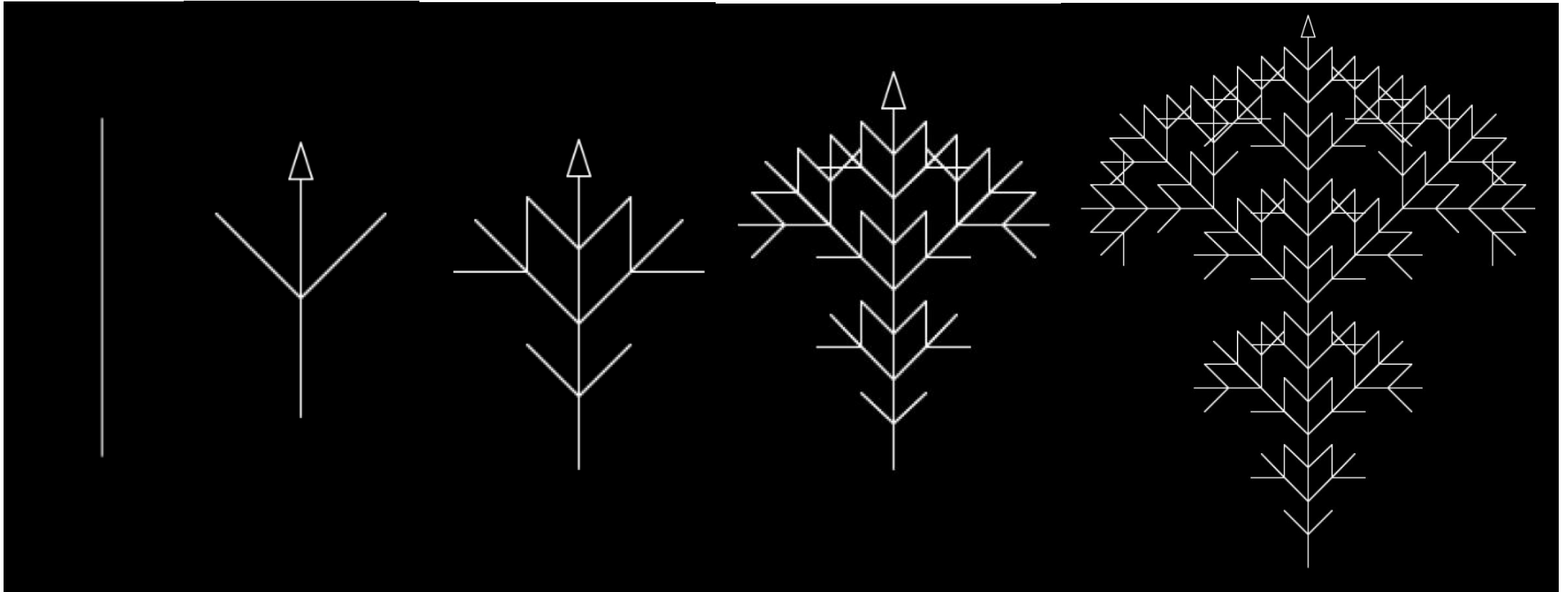
L-Systems

Examples

$$\omega = F$$

$$F \rightarrow F[-F][+F]F$$

Angle 45°



Summary

- **Logo & Turtle-Graphics**
 - What is Logo?
 - Turtle Graphics
 - Base commands
- **Recursion & Fractals**
 - What is recursion?
 - Pythagoras tree
 - Fractals
 - Koch flake
- **L-Systems**
 - Production rules
 - Edge rewriting
 - Node rewriting
 - Branching

Sources

- Hromkovic, Juraj. *Einführung in die Programmierung mit LOGO*. Vol. 206. Vieweg+ Teubner, 2010.
- Solomon, Cynthia, et al. "History of logo." *Proceedings of the ACM on Programming Languages* 4.HOPL (2020)
- <https://fmslogo.sourceforge.io/workshop/getting-started.shtml>
- <https://www.michael-holzapfel.de/themen/grenzwert/koch-schneeflocke/koch-schneeflocke.htm>
- <https://logos-download.com/43072-rwth-aachen-university-logo-download.html>
- <https://www.michael-holzapfel.de/themen/grenzwert/koch-schneeflocke/koch-schneeflocke.htm>
- <https://docs.python.org/3/library/turtle.html>
- <https://www.calormen.com/jslogo/language.html>

Sources

- <https://dl.acm.org/doi/abs/10.1145/3386329>
- <https://algorithmicbotany.org/papers/abop/abop.pdf>
- <https://www.youtube.com/watch?v=54XA0pv--bl>
- <https://de.wikipedia.org/wiki/Pythagoras-Baum>
- http://joachim-wedekind.de/Downloads/LogoBuchBand1_redsize.pdf
- <https://slate.com/technology/2021/06/golden-ratio-phi-irrational-number-ellenberg-shape.html>
- <https://www.veritasnewspaper.org/post/the-golden-ratio-and-its-applications>
- <https://www.houdinikitchen.net/wp-content/uploads/2019/12/L-systems.pdf>